

TECHCATALYST DE 2026

Intermediate Python & APIs

Error handling, files, classes, then your code reaches the internet: APIs as reliable data sources.

Week 2 Day 3

Exceptions done right

Files and JSON

OOP for data models

HTTP and REST

requests, httpx, SDKs

Today's agenda

Time	Block
8:00 - 8:30	Recap and drill review
8:30 - 9:30	Exceptions: fail clearly, recover intentionally
9:45 - 11:00	Files and JSON: local data before remote data
11:00 - 12:00	Classes as records, clients, and connectors
1:00 - 2:00	HTTP and REST: requests, status, JSON, pagination
2:00 - 3:00	httpx, async, SDKs, and key handling
3:00 - 4:30	Independent build: weather API explorer
4:30 - 5:00	Code reading circle and recap

Exceptions: plan for failure

```
try:
    fare = float(raw_value)
except ValueError:
    log_bad_row(raw_value)      # handle what you EXPECT
    fare = None
except Exception as e:
    print(f"unexpected: {e}")  # catch-all LAST, and rarely
    raise                     # re-raise: don't swallow surprises
finally:
    close_connection()        # runs no matter what
```

Pipeline rule: catch specific exceptions you can handle; let everything else crash loudly. A silent except: pass corrupts data for weeks before anyone notices.

Files: text, CSV, JSON

```
import csv, json

with open("zones.csv") as f:                # 'with' closes it for you
    reader = csv.DictReader(f)              # each row = a dict!
    zones = [row for row in reader]

with open("trip.json") as f:
    trip = json.load(f)                    # JSON -> dicts & lists

with open("out.json", "w") as f:
    json.dump(trips, f, indent=2)          # and back again
```

DictReader + json.load: yesterday's dict skills are exactly the file skills. Pandas does this at scale tomorrow.

02

Objects, briefly

Just enough OOP to read real code and model your data.

Classes: bundling data + behavior

```
class TripRecord:
    def __init__(self, vendor, fare, zone):
        self.vendor = vendor
        self.fare = fare
        self.zone = zone

    def is_valid(self):
        return self.fare is not None and 0 < self.fare <= 300

trip = TripRecord("CMT", 18.42, "JFK")
trip.is_valid()          # True
```

self = "this object". Mostly you'll USE classes others wrote , build one today so they stop being magic.

Why DEs meet classes everywhere

Data models

A record type with validation attached (you just wrote one). Libraries like Pydantic industrialize this.

Connectors / clients

`storage.Client()`, `requests.Session()` ,
objects that hold config + connection
state

Frameworks

Airflow DAGs, Beam transforms, Spark
sessions , you subclass or instantiate
theirs

Goal today is reading fluency, not OOP mastery , composition and dunderers can wait.

03

Talking to APIs

APIs are contracts with systems you do not control.

HTTP gives every API call a receipt

Piece	Example	Meaning
Method	GET / POST	Read / create (DEs mostly GET)
URL + params	/v1/trips?zone=JFK&limit=100	What you're asking for
Headers	Authorization: Bearer <key>	Who you are, what format you want
Status code	200 / 404 / 429 / 500	OK / not found / slow down! / their bug
Body	JSON (usually)	Payload your script parses into dicts and lists

Status code first, JSON second. A pipeline should check the receipt before trusting the body.

A safe request has four guardrails

```
import requests

url = "https://api.github.com/search/repositories"
headers = {"User-Agent": "techcatalyst-de-2026"}
params = {"q": "data engineering", "per_page": 5, "page": 1}

resp = requests.get(url, headers=headers, params=params, timeout=10)
resp.raise_for_status()
payload = resp.json()
records = payload["items"]

print(resp.status_code, len(records))
```

params shapes the question; timeout prevents hangs; raise_for_status checks failure; json turns the body into dicts and lists.

Pagination means complete data arrives in chunks

Page

An API returns one window of a larger result set, for example 5 records out of thousands.

Cursor

You request the next window with `page/per_page`, `limit/offset`, or a `next-page` token.

Stop rule

Stop when the page is empty, there is no next token, or your max page guard is reached.

Rate limits are traffic rules

Signal

A 429 or retry header means the service is telling your pipeline to slow down.

Response

Pause, retry with backoff, and cap attempts. Do not loop faster after failure.

Why DEs care

A noisy pipeline can get a key, IP, or team blocked. Polite clients keep ingestion reliable.

requests is the baseline; httpx is the async path

requests

Simple, synchronous, widely used
Great for scripts and quick checks
Common in existing codebases

httpx

Same get, params, headers, json model
Client and AsyncClient options
Modern timeout and connection handling

Decision rule

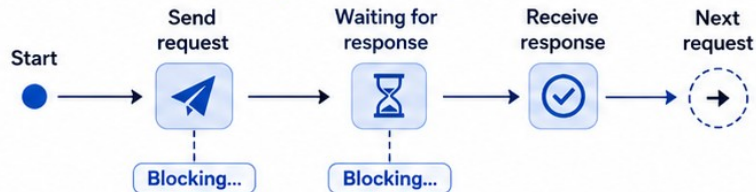
Use requests to learn the baseline
Use httpx for async or modern clients
Keep the polite-client habits either way

Python HTTP Clients: requests vs HTTPX

requests

 Primarily synchronous & blocking

— How it works (one request at a time) —



```
</> import requests  
response = requests.get(url)
```

- ✓ Synchronous by design
- ✓ Simple and widely used
- ✓ Great for scripts and one-off calls
- ✓ Each request blocks until it finishes

 **Best for:**
simple scripts,
notebooks,
quick integrations

HTTPX

 Supports both sync and async

Sync mode

```
</> import httpx  
response = httpx.get(url)
```

 Feels familiar
if you know
requests

Async mode



```
</> import httpx  
async with httpx.AsyncClient() as client:  
    response = await client.get(url)
```

- ✓ Supports sync and async
- ✓ Can handle many I/O calls concurrently
- ✓ Good fit for FastAPI and async apps
- ✓ Modern client with HTTP/2 support

 **Best for:**
high-concurrency apps,
async systems,
scalable API work



Mental model

`requests` = simple blocking calls

`HTTPX` = requests-like API plus async power

Which should I choose?

1



Need the simplest
possible script?

`requests`

2



Need to make many
API calls efficiently?

`HTTPX`

3



Using async Python
(FastAPI, asyncio)?

`HTTPX`

Async overlaps waiting, not computation

```
# sync: one weather call finishes before the next starts
with httpx.Client(timeout=10) as client:
    for city in cities:
        response = client.get(WEATHER_URL, params=city.params)

# async: many calls wait on the network together
async with httpx.AsyncClient(timeout=10) as client:
    tasks = [client.get(WEATHER_URL, params=c.params) for c in cities]
    responses = await asyncio.gather(*tasks)
```

Use async when most time is network waiting. Keep CPU-heavy cleaning in ordinary functions or separate workers.



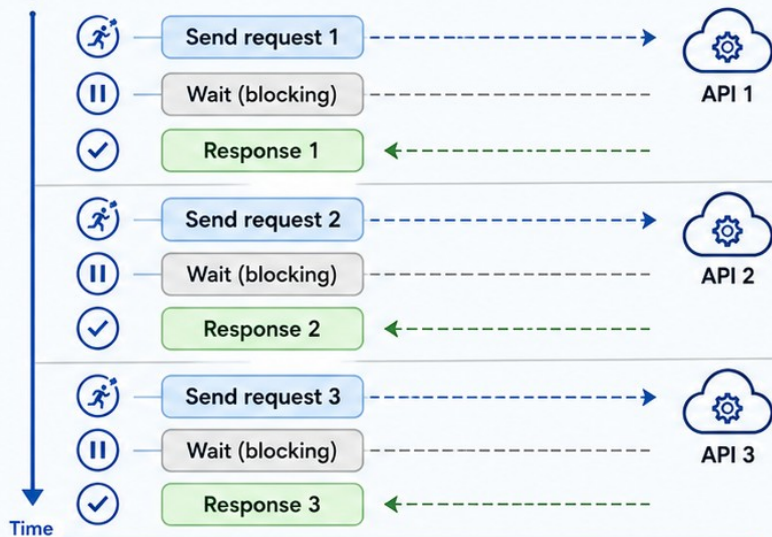
Synchronous vs Asynchronous API Calls



Synchronous



Single Worker / Single Thread (Blocking)



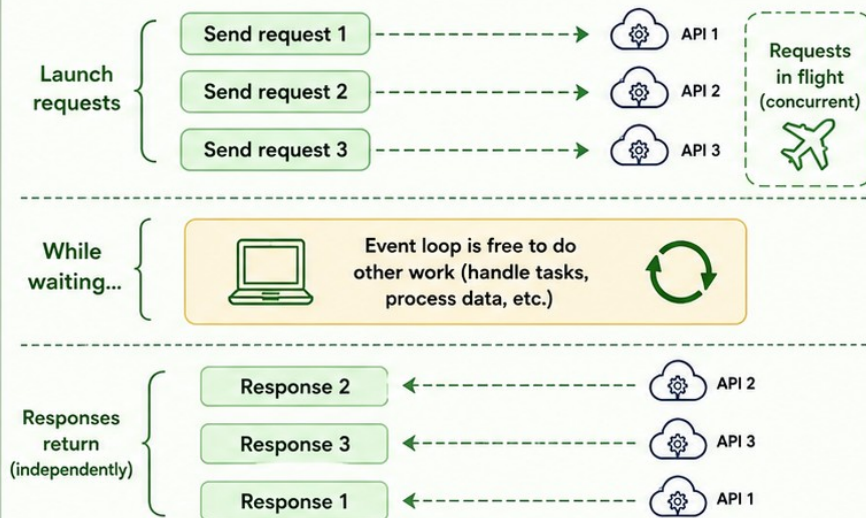
- ✓ One request at a time
- ✓ Blocking while waiting
- ✓ Simple mental model
- ✓ Total time grows with each call



Asynchronous



Event Loop / Async Manager (Non-Blocking)



- ✓ Many requests can be in flight
- ✓ Non-blocking while waiting
- ✓ Best for I/O-heavy workloads
- ✓ Lower total waiting time



If your app spends time waiting on network responses, async can improve throughput.

An SDK wraps the API contract

API

The contract: method, URL, params, headers, body, status

You build and send the request

Works from any language

SDK

A Python wrapper around the API

Handles auth, signing, retries, pagination

boto3 for AWS, google-genai for Gemini

When to use

Use SDKs for cloud services and production clients

Use raw HTTP to learn or for simple public endpoints

Same contract, safer defaults

Hide your keys with .env

```
# .env    (never commit this file)
GOOGLE_API_KEY=your-key-here
AWS_ACCESS_KEY_ID=your-id
AWS_SECRET_ACCESS_KEY=your-secret

# .gitignore
.env

# load it in Python
from dotenv import load_dotenv
import os
load_dotenv()
key = os.environ["GOOGLE_API_KEY"]
```

Keys are credentials, not settings. Load from the environment, keep .env in .gitignore, and rotate anything leaked.

API literacy still matters in the AI era

AI services

Gemini, OpenAI, and Claude use requests, auth, JSON, and errors

You may call them directly or through an SDK

The same failure rules apply

Cloud and data services

AWS, GCP, weather, maps, payments

Your pipelines are API orchestration

SDKs hide syntax, not concepts

Agent tools

MCP and tool calls coordinate APIs underneath

Debugging still needs request and response thinking

The durable skill is reading the contract

Demo: GitHub and Open-Meteo, live

We will inspect two real, keyless APIs together:

1. GitHub repo endpoint: status, headers, and JSON body
2. GitHub search endpoint: params, page, per_page, total_count
3. Open-Meteo forecast endpoint: latitude, longitude, current and hourly fields
4. requests to httpx: same mental model, modern client
5. Retry and backoff shape: protect the API without hammering it
6. Bridge: API JSON becomes weather_records for DataFrames

Independent build: weather API explorer

Goal: build a small CLI by hand: fetch weather, retry transient failure, summarize one city.

Build

1. Finish the GitHub and Open-Meteo notebook; keep your `weather_records` list.
2. Compare sync and async weather calls, then explain when async helps.
3. Turn API records into pandas and Polars DataFrames.
4. Ship `weather_explorer.py` with CLI args, timeout, retry, and summary output.

Deliverable: `weather_explorer.py` plus completed notebooks in `student-work/week2/day3`, committed through your PR.

Key takeaways

- 1 Catch expected exceptions and leave surprises visible. Never except: pass.
- 2 Files and JSON turn external data into auditable local artifacts.
- 3 Classes show up as records, clients, connectors, and framework objects.
- 4 `timeout`, `raise_for_status`, `pagination`, and `retries` make ingestion reliable.
- 5 `httpx`, `async`, `SDKs`, and `.env` key handling are the production-shaped next step.